CSharp gyakorlófeladatok a szemeszter második feléhez

Információábrázolás és algoritmizálás alapjai

Hallgatói feladatlap a 6–14. héthez. Hétről hétre külön konzolprojektben dolgozzon. Először tervezze meg a megoldást, majd egészítse ki a TODO jelölésű részeket, futtassa az ellenőrző példákat, és magyarázza el az eredményt.

Hét	Gyakorlati témakör
6	Karakterkódolás és algoritmusleírás
7	Feladatelemzés és algoritmusterv
8	Összegzés megszámlálás és eldöntés
9	Kiválasztás kiválogatás és szélsőértékek
10	Metszet egyesítés és összefuttatás
11	Lineáris strázsás és rendezett lineáris keresés
12	Bináris keresés és előfeltételei
13	Három egyszerű rendezési algoritmus
14	Összefoglaló eredményfeldolgozás

A munkavégzés szabályai

Az Indulo mappában minden hét külön Program.cs és konzolprojekt. A kész Main és a késznek jelölt segédrészek a kipróbálást szolgálják. A kódváz lefordul, de a helyettesítő visszatérési értékek még nem oldják meg a feladatot. A TODO utáni mintasort a teljes saját algoritmusával helyettesítse.

Az algoritmusokat ciklusokkal és elágazásokkal készítse el. A feldolgozást ne helyettesítse LINQ, Sum, Count, Any, Min, Max, Contains, Intersect, Union, OrderBy, Array.Sort, List.Sort vagy beépített keresőművelettel. A List.Add, a tömbmásolás és a kész megjelenítő műveletek használhatók. A karakterkódolási feladatban az Encoding műveletei kifejezetten szükségesek.

A tömbök és listák nem null értékűek. Az üres bemenet kezelését minden feladat külön rögzíti. Az indexek nullától indulnak; a -1 hiányjelzés, ezért nem használható elem elérésére. A megadott előfeltételeket tartsa be.

Minden héthez készítsen rövid munkajegyzetet: algoritmusterv, legalább három próba, tényleges eredmények és egy megmagyarázott határeset. A témák kétórás gyakorlásra készültek; az önálló bővítés a gyorsabban haladóknak szól.

6 hét Karakterkódolás és algoritmusleírás

Projekt: Het6. Vizsgálja meg ugyanannak a szövegnek a karakterértékeit és bájtos kódolását. Ezután írjon le egy egyszerű döntési algoritmust háromféle módon.

6 1 ASCII tartomány

Egészítse ki az AsciiE függvényt a TODO 6.1 helyén. Akkor adjon igaz értéket, ha a char numerikus értéke 0 és 127 közötti. Vizsgálja meg az A, a 0, az á és az ő karaktert. Írja fel a kész kiírás decimális és U+ jelölésű értékeit.

6 2 UTF 8 bájtmennyiség

A TODO 6.2 helyén az Encoding.UTF8.GetByteCount művelettel számolja meg a szöveg kódolásához szükséges bájtokat. A bájtszámot ne a Length tulajdonságból számítsa ki. Vizsgálja meg az ABC, az ár és az A után U+1F600 kódpontot tartalmazó szöveget. Az utóbbi C# alakja: "A\U0001F600".

6 3 Kódolás és visszaolvasás

Futtassa az ASCII és UTF-8 bemutatót az ár szóval. Jegyezze fel a visszaolvasott szövegeket. A KodpontDarab kész segédfüggvényét most használja változtatás nélkül. Hasonlítsa össze a kódpontok számát, a Length értékét és az UTF-8 bájtszámot.

6 4 Két szám nagyobbika

Írja le szövegesen két egész szám nagyobbikának meghatározását, készítsen folyamatábrát és pszeudokódot, majd egészítse ki a Nagyobbik függvényt. Egyenlőség esetén ugyanaz az érték az eredmény. Jelöljön meg egy konkrét utasítást, nevezze meg az algoritmust, és különítse el a C# programtól.

6 hét Ellenőrzés és önálló gyakorlás

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
A és 0	65 és 48; mindkettő ASCII
á és ő	225 és 337; egyik sem ASCII
ABC	Length 3; kódpont 3; UTF-8 3 bájt
ár	Length 2; kódpont 2; UTF-8 3 bájt
A + U+1F600	Length 3; kódpont 2; UTF-8 5 bájt
Üres szöveg	Mindhárom számláló 0
7 és 3; 4 és 4	Nagyobbik: 7; 4

Magyarázza meg az eredményt

1. Miért nem jelenti a char típusa azt, hogy minden Unicode kódpont egyetlen char értékben elfér?
2. Az ASCII kódolás utáni visszaolvasás helyreállítja-e az elveszett á betűt?
3. Miben azonos a szöveges leírás, a folyamatábra és a pszeudokód? Miben különbözik a formájuk?

Önálló bővítés

Írassa ki az UTF-8 bájtokat hexadecimálisan a kész BitConverter.ToString segítségével. Vizsgálja meg az ő karaktert is.

7 hét Feladatelemzés és algoritmusterv

Projekt: Het7. Minden programozási rész előtt írja fel a bemenetet, a kimenetet, az érvényes tartományt és legalább három határesetet. Az elágazások sorrendjét előbb pszeudokóddal tervezze meg.

7 1 Pontszám minősítése

A pontszám egész szám. Érvényes tartománya 0–100; 50 ponttól Sikeres, alatta Sikertelen az eredmény. Tartományon kívül Érvénytelen szöveget adjon vissza. Készítse el a PontMinosites függvényt. A Main végén a TryParse kész bemutató: szöveges hiba esetén külön üzenetet ír ki.

7 2 Rendelés végösszege

A termékek összára 0–1 000 000 Ft közötti egész szám. 10 000 Ft alatt a szállítás 1500 Ft, ettől kezdve ingyenes. A Vegosszeg a teljes fizetendő összeget adja vissza; érvénytelen bemenetnél -1-et. A feladat szabálya szerint a 0 Ft-os rendeléshez is 1500 Ft szállítás tartozik.

7 3 Percek felbontása

Az IdoSzoveg függvény 0–1 000 000 közötti egész percértéket kap. Adja vissza óra:perc alakban; a perc mindig két számjegyű legyen. Használjon egész osztást és maradékképzést. A tartományon kívüli értékek kezelése ennél a függvénynél nem feladat.

7 4 A terv ellenőrzése

A 7.2 feladatot írja le természetes nyelven és pszeudokóddal. Azonosítsa az érvényességvizsgálatot és az árképzési döntést. Készítsen táblázatot bemenet, várt eredmény, tényleges eredmény oszlopokkal. Mutassa meg, miért hibás a pont >= 50 vizsgálatot az érvényesség ellenőrzése elé tenni.

7 hét Ellenőrzés és önálló gyakorlás

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
49; 50; 100 pont	Sikertelen; Sikeres; Sikeres
-1; 101 pont	Érvénytelen; Érvénytelen
alma a konzolbemeneten	Hibás számformátum.
0; 9999 Ft	1500; 11499 Ft
10000; 1000000 Ft	10000; 1000000 Ft
-1; 1000001 Ft	-1; -1
0; 59; 60; 125 perc	0:00; 0:59; 1:00; 2:05

Magyarázza meg az eredményt

1. Melyik pontszám mutatja meg, hogy a hibás feltételsorrend érvénytelen adatot fogad el?
2. Miért szükséges a 9999 és a 10000 Ft-os próbát is elvégezni?
3. Milyen különbség van egy függvény előfeltétele és az általa visszaadott hibajelzés között?

Önálló bővítés

Egészítse ki a pontszám minősítését érdemjeggyel: 0–49 elégtelen, 50–62 elégséges, 63–74 közepes, 75–86 jó, 87–100 jeles. Az érvényességvizsgálat maradjon meg.

8 hét Összegzés megszámlálás és eldöntés

Projekt: Het8. Dolgozzon a 12, -3, 0, 7, 12, -8, 5 sorozattal. Az összegzést és az eldöntést tömbön, a megszámlálást List<int> listán valósítsa meg. Minden bemenet nem null értékű; az üres sorozat megengedett.

8 1 Összegzés tömbön

Az Osszeg függvény ciklussal adja össze az elemeket. Az akkumulátor és a visszatérési érték long típusú legyen. Üres sorozat összege 0. Az összeget ne a sorozat első elemével kezdje.

8 2 Feltételes megszámlálás listán

A PozitivDarab függvény számolja meg a nullánál nagyobb elemeket. A 0 nem pozitív. A List<int> elemszámát a Count adja meg; az elemeket ugyanúgy nullától indexeljük, mint a tömbben.

8 3 Eldöntés korai leállással

A VanNegativ akkor adjon igazat, ha van negatív elem. A ciklus az első találatnál álljon meg. A feltételben előbb a tömbhatárt, utána az elemet vizsgálja. Üres sorozatnál az eredmény hamis.

8 4 Átlag és algoritmuskövetés

A kész Main az összeget double értékké alakítja az osztás előtt. Magyarázza el ennek szerepét és az üres sorozat külön kezelését. Készítsen i, aktuális elem, részösszeg táblázatot az összegzéshez. Futtassa az alappróbán kívül az üres, a csupa nulla és a csupa negatív sorozatot.

8 hét Ellenőrzés és önálló gyakorlás

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
Alapsorozat	Összeg 25; pozitív 4; van negatív: igaz
Alapsorozat átlaga	25 / 7, közelítőleg 3,57142857
Üres sorozat	Összeg 0; pozitív 0; negatív nincs; nincs átlag
0, 0, 0	Összeg 0; pozitív 0; negatív nincs
-2, -1	Összeg -3; pozitív 0; van negatív
2147483647, 1	Összeg 2147483648, long típussal

Magyarázza meg az eredményt

1. Melyik algoritmusnak kell mindig minden elemet feldolgoznia?

2. Miért veszélyes felcserélni az i < a.Length és az a[i] >= 0 feltételeket?

3. Mikor adhat hamis eredményt az eldöntés: az első nem megfelelő elemnél vagy a teljes sikertelen keresés után?

Önálló bővítés

Írjon külön függvényt a páros elemek megszámlálására és annak eldöntésére, van-e 100-nál nagyobb elem. Használja a korábbi algoritmusok szerkezetét.

9 hét Kiválasztás kiválogatás és szélsőértékek

Projekt: Het9. Az alapsorozat: 7, 2, 9, 2, -4, 12, 5. A függvények indexei nullától indulnak. Azonos szélsőértékek esetén az első előfordulás helyét őrizze meg.

9 1 Kiválasztás és előfeltétel

A KivalasztParosIndex a sorozat első páros elemének indexét adja vissza. Előfeltétele, hogy legyen páros elem. A kész ElsoParosIndex hívófüggvény ezt előbb ellenőrzi, és hiány esetén -1-et ad. A feltétel nélküli kiválasztást ne hívja közvetlenül üres vagy csupa páratlan sorozatra.

9 2 Kiválogatás két feltétellel

A PozitivParosak új List<int> listába gyűjtse a pozitív és páros elemeket. Az eredeti sorrend és az ismétlődések maradjanak meg. A két feltételt ÉS kapcsolat kösse össze. A bemeneti tömb ne változzon.

9 3 Minimumhely meghatározása

A MinIndex adja vissza a legkisebb elem első indexét. Üres bemenetre -1 legyen az eredmény. Nem üres esetben a kezdő jelölt az első elem helye; a keresés a második elemmel folytatódjon.

9 4 Maximumhely meghatározása

A MaxIndex ugyanilyen szabályokkal keresse a legnagyobb elemet. A kiírás előtt ellenőrizze, hogy a kapott index nem -1. A szélsőértéket a visszaadott indexen érje el.

9 hét Ellenőrzés és önálló gyakorlás

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
Alapsorozat első párosa	Index 1; érték 2
Pozitív párosak	2, 2, 12
Minimum és maximum	Minimum -4, index 4; maximum 12, index 5
1, 3, 5	ElsoParosIndex: -1; kiválogatás: üres
7, 2, 9, 2	Első minimum indexe 1
Üres sorozat	ElsoParosIndex, MinIndex és MaxIndex: -1
5	Minimum- és maximumindex: 0

Magyarázza meg az eredményt

1. Mi a különbség a kiválasztás egyetlen indexe és a kiválogatás eredménylistája között?
2. Miért hibás a minimum aktuális értékét minden bemenetre 0-val kezdeni?
3. Hogyan változik az első minimum helye, ha < helyett <= szerepel a frissítésben?

Önálló bővítés

Válogassa ki a [0, 10] intervallumba eső páros elemeket. Írja fel külön a három feltételt, majd vonja össze őket.

10 hét Metszet egyesítés és összefuttatás

Projekt: Het10. Különítse el a halmazműveleteket a rendezett sorozatok összefuttatásától. A metszet és az egyesítés ismétlődésmentes eredményt ad; az összefuttatás az összes előfordulást megtartja.

10 1 Saját tartalmazásvizsgálat

A Tartalmaz függvény ciklussal keressen egy List<int> listában. Találatnál igaz, a teljes sikertelen bejárás után hamis legyen az eredmény. Ezt a segédfüggvényt használja a következő két feladatban.

10 2 Metszet

Bemenet: A = [3, 1, 3, 5], B = [3, 4, 4, 1]. A Metszet csak a mindkét bemenetben szereplő különböző értékeket adja vissza. A sorrend az A-beli első előfordulás rendje legyen. A bemeneti ismétlődéseket kezelje saját tartalmazásvizsgálattal.

10 3 Egyesítés

Az Egyesites gyűjtse össze az összes különböző értéket. Előbb A elemeit dolgozza fel eredeti sorrendben, majd B még nem szereplő elemeit. Az eredményben minden érték egyszer jelenjen meg.

10 4 Rendezett sorozatok összefuttatása

Bemenet: X = [1, 3, 3, 8], Y = [2, 3, 9]. Két indexszel mindig a kisebb következő elemet vegye át. Egyenlőség esetén X eleme következzen. A fő ciklus után másolja át a megmaradt elemeket. Mindkét bemenet előfeltétele a nemcsökkenő rendezettség.

10 hét Ellenőrzés és önálló gyakorlás

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
A és B metszete	3, 1
A és B egyesítése	3, 1, 5, 4
X és Y összefuttatása	1, 2, 3, 3, 3, 8, 9
Metszet üres bemenettel	Üres lista
Egyesítés: [], [2, 2, 1]	2, 1
Összefuttatás: [], [1, 1]	1, 1
Összefuttatás: [], []	Üres lista

Magyarázza meg az eredményt

1. Miért nem elegendő a metszethez csak a B-ben való előfordulást vizsgálni?
2. Miért szerepel három darab 3-as az összefuttatás eredményében?
3. Mi történne, ha a fő ciklus után elmaradna a maradék elemek átmásolása?

Önálló bővítés

Készítse el A és B különbségét: csak azok az A-beli értékek kerüljenek az eredménybe, amelyek B-ben nincsenek. Az eredmény legyen ismétlődésmentes.

11 hét Lineáris strázsás és rendezett lineáris keresés

Projekt: Het11. Bemenet: 2, 5, 8, 12, 17, 23, 31. A keresések a találat első indexét adják; hiány esetén -1-et. A mérőszám a megvizsgált elempozíciók száma, nem a feltételben szereplő összes összehasonlítás száma.

11 1 Lineáris keresés

A Linearis járja be a tömböt balról jobbra. Minden megvizsgált elemnél növelje a vizsgalt számlálót. Találatnál térjen vissza az indexszel. A keresés rendezetlen bemeneten is működjön.

11 2 Strázsás keresés

A Strazsas készítsen egy eggyel hosszabb másolatot, és az utolsó helyre írja a keresett értéket. A keresőciklus csak az egyenlőséget vizsgálja. Az eredeti hossz alatti találat sikeres, a strázsa indexénél -1 a válasz. A vizsgált pozíciók közé a strázsa helye is számítson.

11 3 Rendezett lineáris keresés

A RendezettLinearis előfeltétele a nemcsökkenően rendezett tömb. Találatnál adja vissza az indexet. Ha az aktuális elem már nagyobb a keresettnél, adjon vissza -1-et. A számláló csak egyszer növekedjen egy feldolgozott pozícióhoz.

11 4 Módszerek összehasonlítása

Keresse a 2, 17, 7 és 40 értékeket mindhárom módszerrel. Hasonlítsa össze az indexeket és a számlálókat. Az out paraméter kész hívása a második eredményt veszi át; ehhez a feladathoz az átadás szabályait elég a minta szerint használni.

11 hét Ellenőrzés és önálló gyakorlás

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
2 keresése	Index 0; vizsgált pozíciók: 1 / 1 / 1
17 keresése	Index 4; vizsgált pozíciók: 5 / 5 / 5
7 keresése	Index -1; vizsgált pozíciók: 7 / 8 / 3
40 keresése	Index -1; vizsgált pozíciók: 7 / 8 / 7
Üres tömb	Index -1; vizsgált pozíciók: 0 / 1 / 0
2, 2, 5 sorozatban 2	Mindhárom módszer indexe 0

Magyarázza meg az eredményt

1. Mi garantálja a strázsás keresőciklus végét határvizsgálat nélkül?
2. Miért nem bizonyítja a kisebb számláló önmagában a teljes program gyorsabb futását?
3. Miért nem szabad a rendezett lineáris keresés korai kilépését rendezetlen tömbre alkalmazni?

Önálló bővítés

Rajzolja fel a három keresés által megvizsgált pozíciókat a hiányzó 7 keresésekor. Jelölje külön a strázsa új helyét.

12 hét Bináris keresés és előfeltételei

Projekt: Het12. Használja a 2, 5, 8, 12, 17, 23, 31 rendezett List<int> listát. A bináris keresés a megvizsgált középső pozíciókat számolja, találat hiányában -1-et ad.

12 1 Rendezettség ellenőrzése

A RendezettE hasonlítsa össze a szomszédos elemeket. Egyetlen csökkenő párnál hamis legyen az eredmény; a teljes ellenőrzés után igaz. Az üres és az egyelemű lista rendezettnek számít, az egyenlő szomszédok megengedettek.

12 2 Bináris keresés

A Binaris a bal = 0, jobb = Count - 1 határokkal induljon. Amíg bal <= jobb, vizsgálja a középső elemet. A következő intervallumból a már megvizsgált középső pozíció maradjon ki. Találatnál adja vissza az indexet. Ismétlődő értékeknél bármely megfelelő index elfogadható.

12 3 Kézi algoritmuskövetés

A 17, majd a 7 keresésekor készítsen bal, jobb, kozep, a[kozep] táblázatot. Futtatás előtt írja fel a várt eredményt. A Debugger segítségével ellenőrizze az intervallum változását.

12 4 Hibás alkalmazás elemzése

A [8, 2, 17, 5] lista rendezetlen. Kézzel kövesse a bináris keresést a 8-ra, és mutassa meg, miért veszíti el a találatot. A kész Main ezen a bemeneten nem hívja meg a keresést: előbb a rendezettséget ellenőrzi. Magyarázza el a bal = kozep helyett szükséges bal = kozep + 1 szerepét.

12 hét Ellenőrzés és önálló gyakorlás

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
17; 7 keresése az alaplistán	Index 4; -1. Mindkettő 3 középvizsgálat.
2; 31 keresése	Index 0; 6. Mindkettő 3 középvizsgálat.
Üres lista	Rendezett; találat -1; 0 középvizsgálat
[5] listában 5; 4	Index 0, 1 vizsgálat; -1, 1 vizsgálat
[2, 2, 2] listában 2	Bármely 0–2 index, a jelen kódban 1
[8, 2, 17, 5]	Rendezettség hamis; a hívó elutasítja

Magyarázza meg az eredményt

1. Hány elemet tartalmaz a keresési intervallum, amikor bal > jobb?
2. Miért lehet végtelen a ciklus, ha bal = kozep frissítés marad a kódban?
3. Miért nem érdemes minden keresés előtt újra ellenőrizni egy változatlan lista rendezettségét?

Önálló bővítés

Hasonlítsa össze a lineáris és a bináris keresést egy 1–64 értékeket tartalmazó listán. Keresse a 64-et és a hiányzó 65-öt. A rendezettség ellenőrzését külön számolja.

13 hét Három egyszerű rendezési algoritmus

Projekt: Het13. Minden algoritmus nemcsökkenő sorrendbe rendezze ugyanannak a bemenetnek egy külön másolatát. A Clone kész hívása másolatot ad; egyszerű értékadással csak ugyanazt a tömböt érné el két néven.

13 1 Buborékrendezés

Írja meg a szomszédos elemek összehasonlítását és cseréjét. Minden menetben csökkenjen a vizsgált rész vége. Ha egy teljes menetben nem történt csere, a rendezés fejeződjön be. Számolja az elemértékek összehasonlításait és a tényleges cseréket.

13 2 Kiválasztásos rendezés

A rendezetlen rész minimumának helyét keresse meg, majd az aktuális helyre cserélje. Csak eltérő indexek esetén cseréljen. Számolja a minimumkeresés elemérték-összehasonlításait és a tényleges cseréket.

13 3 Beszúró rendezés

A következő kulcselemnek keressen helyet a rendezett kezdőszeletben. A nagyobb elemeket tolja egy hellyel jobbra. Számolja az a[j] és a kulcs közötti összehasonlításokat és a jobbra tolásokat. A kulcs végső beírása ne számítson eltolásnak.

13 4 Összehasonlító vizsgálat

Használja az [5, 1, 4, 2], [1, 2, 4, 5], [5, 4, 2, 1] és [2, 2, 1, 1] bemeneteket. Jegyezze fel a rendezett eredményt és a számlálókat. A ciklushatár-vizsgálatok nem tartoznak az összehasonlítási mérőszámhoz. A csere és az eltolás eltérő művelet; ne vonja őket össze egyetlen futásidő-adattá.

13 hét Ellenőrzés és önálló gyakorlás

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
[5, 1, 4, 2] rendezve	[1, 2, 4, 5] mindhárom eljárásnál
Ugyanez buborékkal	6 érték-összehasonlítás; 4 csere
Ugyanez kiválasztással	6 érték-összehasonlítás; 2 csere
Ugyanez beszúrással	6 érték-összehasonlítás; 4 eltolás
Rendezett bemenet	Összehasonlítások 3 / 6 / 3; mozgatások 0
Üres és egyelemű tömb	Változatlan; minden számláló 0

Magyarázza meg az eredményt

1. Melyik eljárás használja ki a már rendezett bemenetet ebben a megvalósításban?
2. Miért kell mindhárom rendezésnek az eredeti bemenet külön másolatát átadni?
3. Miért nem általános teljesítménymérés egyetlen négyelemű példa számlálója?

Önálló bővítés

Az [5, 1, 4, 2] sorozaton írja fel minden külső menet utáni teljes tömböt. Beszúrásnál jelölje a rendezett kezdőszeletet.

14 hét Összefoglaló eredményfeldolgozás

Projekt: Het14. Készítsen eredménykimutatást két csoport pontszámaiból. A = [62, 48, 100, 75, 62, 0], B = [50, 62, 90, 0]. Minden pont 0–100 közötti; 50-től sikeres az eredmény. A program több korábban megírt algoritmust használ együtt.

14 1 Alapstatisztika

A TODO 14.1a–c részekben készítse el az összegzést, a sikeres eredmények megszámlálását és annak eldöntését, van-e sikertelen pontszám. Az átlag kész kiírása az összeget double értékké alakítja. Üres bemenetre ne történjen nullával osztás.

14 2 Kiválogatás és szélsőértékek

A TODO 14.2a–c részekben adja vissza az első minimum és maximum indexét, valamint a sikeres pontszámok listáját eredeti sorrendben, az ismétlődéseket megtartva. Üres sorozat szélsőértékindexe -1 legyen.

14 3 Két csoport összevetése

A TODO 14.3a–b részekben határozza meg a közös különböző pontértékeket és az egyesített ponthalmazt. Használja a 10. hét sorrendi szabályait. A kész Tartalmaz segédfüggvény használható.

14 4 Rendezés és keresés

A TODO 14.4a helyén beszúró rendezéssel rendezze A másolatát, majd a TODO 14.4b helyén binárisan keresse a 75 és a 49 értéket. Az index a rendezett másolatra vonatkozik. Az eredeti sorrendű A tömb maradjon meg.

14 hét Ellenőrzés és önálló gyakorlás

14 5 Rendezett eredmények összefuttatása

Rendezze B másolatát is, majd a TODO 14.5 helyén futtassa össze a két rendezett tömböt. Minden pontelőfordulás maradjon meg. A két csoport együttes elemszáma egyezzen meg az eredmény hosszával.

14 6 Megfigyelés és ZH próba

Magyarázza el a kész állapotkódot: a 0. bit jelzi, hogy A nem üres; az 1. bit jelzi, hogy van sikertelen eredmény. Értelmezze a kurzusnév UTF-8 bájtmennyiségét és a két ponttömb nyers adatmennyiségét. Önálló próbában végezze el a feladatot üres, majd egyelemű sorozatokkal is.

Ellenőrző példák

Bemenet vagy vizsgálat	Elvárt eredmény
A statisztikája	Összeg 347; sikeres 4; átlag 57,83 két tizedesre
Szélsőértékek	Minimum 0, index 5; maximum 100, index 2
Sikeres pontszámok	62, 100, 75, 62
Metszet; egyesítés	[62, 0]; [62, 48, 100, 75, 0, 50, 90]
Rendezett A; keresés	[0, 48, 62, 62, 75, 100]; 75 indexe 4; 49: -1
Összefuttatás	0, 0, 48, 50, 62, 62, 62, 75, 90, 100
Korábbi hetek megfigyelései	Alapok ő: 9 UTF-8 bájt; állapot 00000011; nyers adat 40 bájt

Magyarázza meg az eredményt

1. Miért veszítjük el az eredeti helyeket, ha A-t közvetlenül rendezzük?
2. Mi a különbség az egyesített ponthalmaz és az összefuttatott pontsorozat között?
3. Miért csak a nyers int elemek mérete a 40 bájt, és nem a teljes futó program memóriaigénye?

Önálló bővítés

Végezzen el külön futást A = [] és B = [] bemenetekkel, majd A = [50], B = [50] adatokkal. Mindkét esethez írja le előre az elvárt statisztikát és halmazeredményeket.